

IPL Win Predictor

Product Requirements Document

Pankaj Kumar · IEST Shibpur · ML Portfolio Project · June 2026

Project	IPL Win Probability Predictor
Type	ML Project + Streamlit Web App
Data Source	Cricsheet.org (free, no signup)
ML Models	Logistic Regression + XGBoost + SHAP
Interface	Streamlit app (pure Python, no frontend coding)
Deployment	Streamlit Community Cloud (100% free)
Timeline	6 weeks

1. What Is This Project?

IPL Win Predictor is a machine learning project that predicts the win probability of an IPL team at any point during a match. You input the batting team, bowling team, venue, current score, overs bowled, and wickets lost — and the model tells you the probability of winning.

The project uses 10+ years of real IPL ball-by-ball data from Cricsheet.org. It is built entirely in Python and deployed as a Streamlit web app — no React, no database, no backend server needed.

2. Why This Project — Interview Answer

This section is what you say when an interviewer asks 'why did you build this?' — have this memorised.

What	Why It Matters to Interviewers
Real data, not Kaggle tutorial	Cricsheet provides raw ball-by-ball JSON files — you write the parsing code yourself. This shows data engineering skill, not just <code>model.fit()</code> .
Feature engineering story	Raw data has balls and runs. You engineer: current run rate, required run rate, wickets in hand, pressure index, venue factor — each one you can explain.
Indian context	Every Indian interviewer follows IPL. You can say 'let's predict tonight's match' during the demo — it becomes a conversation not a presentation.
SHAP explainability	You add SHAP charts showing which features drive each prediction. Almost no fresher does this. It shows you think beyond accuracy scores.
Compares two models	You train Logistic Regression AND XGBoost, compare them, and explain why XGBoost wins on this data. That's ML thinking, not tutorial following.

3. What the App Does (Features)

ID	Priority	Feature	Description
F-01	Must	Match State Input	User selects batting team, bowling team, city/venue, target score, current score, overs, wickets lost via dropdowns and sliders.
F-02	Must	Win Probability Output	App shows win probability as a big percentage with a colour-coded gauge (green > 60%, amber 40–60%, red < 40%).
F-03	Must	Model Comparison Panel	Side-by-side accuracy table showing Logistic Regression vs XGBoost — accuracy, precision, recall, F1.
F-04	Must	SHAP Feature Importance Chart	Bar chart showing which features (runs left, wickets, overs) most influenced the current prediction.
F-05	Must	Historical Head-to-Head Stats	Simple table showing win % of batting team vs bowling team from historical data.
F-06	Should	Probability Over Time Chart	Line chart showing how win probability changed over the innings ball by ball for a selected historical match.
F-07	Nice	Venue Win Rate Chart	Bar chart showing which teams win most often at the selected venue.

4. What You Are NOT Building (Keep It Simple)

This section exists so you don't waste time. These things are out of scope:

- No user login or authentication — not needed, Streamlit is public
- No database — all data loaded from CSV files at app startup
- No React or JavaScript — Streamlit handles the entire UI in Python
- No paid APIs or services — Cricsheet is free, Streamlit Cloud is free
- No real-time live match data — historical data is enough for the portfolio
- No mobile app — Streamlit is responsive enough on desktop

5. What Does 'Done' Look Like?

ID	Acceptance Criterion
G-01	Model accuracy $\geq$ 75% on test set (IPL is unpredictable — 75% is realistic and honest)
G-02	XGBoost outperforms Logistic Regression — you can explain why with numbers
G-03	SHAP chart renders correctly for any input combination
G-04	App deployed on Streamlit Cloud with a public shareable URL
G-05	GitHub repo has clean README with screenshots and how-to-run instructions
G-06	You can demo the entire app and explain every decision in under 5 minutes

6. Week-by-Week Plan

Week	Focus	What You Actually Do
Week 1	Data Collection & Cleaning	Download Cricsheet ZIP files. Write Python script to parse JSON into a clean CSV. Understand the columns — innings, ball, runs, wickets, over.
Week 2	Feature Engineering	Create new columns: <code>current_run_rate</code> , <code>required_run_rate</code> , <code>balls_left</code> , <code>wickets_in_hand</code> , <code>runs_left</code> . This is the most important week — document each feature with a comment explaining why you chose it.
Week 3	EDA (Exploratory Analysis)	Use matplotlib/seaborn to plot: win % by venue, win % batting first vs second, toss impact, score distribution. Save all charts — they go in your README.
Week 4	Model Training & Evaluation	Train Logistic Regression and XGBoost. Print classification report for both. Choose XGBoost as final model. Add SHAP explainer. Save model as <code>ipl_model.pkl</code> with joblib.
Week 5	Streamlit App	Build the 4-page Streamlit app: Predict page, Model Comparison page, Head-to-Head page, EDA Insights page. All pure Python — no HTML.
Week 6	Deploy + Polish	Push to GitHub. Deploy on Streamlit Community Cloud (free, 2 clicks). Write README. Add demo GIF using LICECAP (free). Update resume.